

SPRINGBOOT ANNOTATION SIMPLIFIED



@jeevamuthu



WHAT IS SPRINGBOOT?

- A **Java-based framework** that streamlines the creation of production-ready applications.
- Provides built-in support for **REST APIs, security, and seamless database integration.**
- Popular choice for building **microservices and enterprise-grade systems.**

Standard Application Layers:

- **Controller Layer:** Handles HTTP requests and responses (@RestController).
- **Service Layer:** Implements business logic and workflows (@Service).
- **Repository Layer:** Manages data access and persistence (@Repository).
- **Model Layer:** Defines domain entities and data models (@Entity).

Spring Boot promotes a clean, modular architecture by separating concerns across these layers—making development, testing, and maintenance simpler.



SPRINGBOOT ANNOTATION

1. @SpringBootApplication

Entry point for Spring Boot apps

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

2. @RestController

Creates RESTful APIs (combines @Controller + @ResponseBody).

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```



SPRINGBOOT ANNOTATION

3. @GetMapping / @PostMapping

Map HTTP requests to controller methods.

```
@GetMapping("/users")
public List<User> getUsers() { ... }

@PostMapping("/users")
public User createUser(@RequestBody User user) { ... }
```

4. @Service

Defines a service layer class for business logic.

```
@Service
public class UserService {
    public String processUser() { return "Accepted"; }
}
```



SPRINGBOOT ANNOTATION

5. @Repository

Marks a repository for DB operations

```
@Repository  
public interface UserRepo extends JpaRepository<User, Long> {}
```

6. @Autowired

Injects dependencies automatically.

```
@Service  
public class OrderService {  
    @Autowired  
    private UserRepo userRepo;  
}
```



SPRINGBOOT ANNOTATION

7. @Entity

Maps a Java class to a DB table

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
}
```

8. @Transactional

Ensures a method runs in a database transaction.

```
@Transactional
public void updateBalance(Long id, Double amt) { ... }
```



SPRINGBOOT ANNOTATION

9. @Component

Generic stereotype annotation for Spring-managed beans.

```
@Component  
public class UtilityClass { ... }
```

10. @Bean

Defines a bean to be managed by Spring container.

```
@Bean  
public RestTemplate restTemplate() {  
    return new RestTemplate();  
}
```



SPRINGBOOT ANNOTATION

11. @Configuration

Declares a class as a source of Spring bean definitions.

```
@Configuration
public class AppConfig {
    @Bean
    public PasswordEncoder encoder() {
        return new BCryptPasswordEncoder();
    }
}
```

12. @Value

Injects values from application.properties.

```
@Value("${app.name}")
private String appName;
```



SPRINGBOOT ANNOTATION

13. @Scheduled

Run tasks at fixed intervals.

```
@Scheduled(fixedRate = 5000)
public void runTask() { ... }
```

14. @Cacheable

Cache frequently used results.

```
@Cacheable("users")
public User getUserById(Long id) { ... }
```



The image features a solid blue background. In the top right corner, there is a decorative pattern of concentric, nested chevrons or arrowheads pointing towards the center. In the bottom left corner, there is a pattern of parallel diagonal lines. The main text is centered and reads:

**SHARE YOUR
THOUGHTS AND
FEEDBACK IN
COMMENTS BELOW!**

LIKE | SHARE | SAVE